

LocalConnect

Community Super-App

Full Implementation Master Plan · For AI Agent Build Instructions

8

Core Modules

3

Launch Phases

40+

Features

3

Revenue Streams

Version 1.0 · Complete build plan from zero to production · 2025

Table of Contents

1.	Project Overview & Vision
2.	Full System Architecture
3.	Database Schema Design
4.	API Architecture & Endpoints
5.	Module 1 — Local Marketplace (Grocery & Retail)
6.	Module 2 — Food & Restaurant Ordering
7.	Module 3 — Hotels, Rooms & Accommodation
8.	Module 4 — Hall & Event Venue Booking
9.	Module 5 — Event Planning & Coordination
10.	Module 6 — Home Services (Plumber, Electrician, etc.)
11.	Module 7 — Transport & Delivery Engine
12.	Module 8 — Beauty, Health & Wellness
13.	Three User Apps (Customer / Vendor / Driver)
14.	Real-Time Tracking System
15.	Payment Gateway Integration
16.	Notifications & Communication System
17.	Admin Dashboard
18.	Security & Authentication
19.	Phase 1 — MVP Build Plan (Months 1–4)
20.	Phase 2 — Growth Build Plan (Months 5–9)
21.	Phase 3 — Scale & AI (Month 10+)
22.	Tech Stack & Infrastructure
23.	Revenue Model & Business Logic
24.	AI Agent Build Instructions (Prompt Guide)
25.	Testing, QA & Launch Checklist

1. Project Overview & Vision

LocalConnect is a community super-app that connects every type of local business — grocery shops, restaurants, hotels, wedding halls, home service professionals, beauty parlors, and more — to customers via a single mobile and web platform with integrated delivery, real-time tracking, and online payments.

Core Problem Being Solved

- Housewives and families cannot easily order from small local shops online
- Finding a plumber or electrician urgently is time-consuming and unreliable
- Booking a wedding hall or event venue requires many phone calls with no transparency
- Local businesses have no digital storefront or order management system
- No single platform covers all daily local needs — grocery, food, services, stays
- Transportation and delivery for orders from small shops does not exist locally

Vision Statement

Build the Talabat + Uber + UrbanClap + Airbnb of local communities — one app that handles every daily need, powered by a local delivery network and real-time technology.

Target Users

User Type	What They Do	Key Benefit
Customers / Residents	Order, book, hire, and track	One app for everything
Shop & Restaurant Owners	List products, receive orders	Go digital instantly
Hotels & Hall Owners	Manage bookings, availability	Zero-commission alternative
Home Service Pros	Receive job requests nearby	More clients, no middleman
Delivery Drivers / Riders	Accept and deliver orders	Flexible income by km
Event Planners	Bundle vendors for events	Manage entire event in app

2. Full System Architecture

Architecture Type: Microservices + Monorepo

The system is split into loosely-coupled services communicating via REST APIs and message queues. A shared monorepo manages all services for easier development.

Frontend Layer

App / Platform	Technology	Purpose
Customer Mobile App	React Native (Expo)	iOS + Android from one codebase
Vendor Mobile App	React Native (Expo)	Shop owners manage orders
Driver Mobile App	React Native (Expo)	Drivers accept & track deliveries
Customer Web App	Next.js 14 (App Router)	Full web access + SEO
Vendor Web Dashboard	Next.js 14	Desktop order & product mgmt
Admin Panel	Next.js 14 + shadcn/ui	Full platform control

Backend Services

Service Name	Technology	Responsibility
API Gateway	Nginx + Node.js	Route all requests, rate limiting
Auth Service	Node.js + JWT	Login, register, token refresh
User Service	Node.js + PostgreSQL	User profiles, addresses
Vendor Service	Node.js + PostgreSQL	Shop profiles, menus, inventory
Order Service	Node.js + PostgreSQL	Create, track, manage orders
Delivery Service	Node.js + Redis	Driver dispatch, live tracking
Booking Service	Node.js + PostgreSQL	Hotels, halls, appointments
Service Request Engine	Node.js + PostgreSQL	Home services job matching
Payment Service	Node.js + Stripe/PayHere	Process payments, refunds
Notification Service	Node.js + Firebase	Push, SMS, email alerts
Search Service	Elasticsearch	Full-text shop/product search
Review Service	Node.js + PostgreSQL	Ratings and reviews
Analytics Service	Node.js + ClickHouse	Business & platform analytics
Media Service	Node.js + AWS S3	Image upload and serving

Infrastructure & DevOps

Component	Tool / Service	Notes
Primary Database	PostgreSQL 16	All transactional data
Cache Layer	Redis 7	Sessions, live tracking, queues
Search Engine	Elasticsearch 8	Product and vendor search
File Storage	AWS S3 / DigitalOcean Spaces	Images, documents
Realtime Engine	Socket.io + Redis PubSub	Live order and driver tracking
Message Queue	BullMQ (Redis-backed)	Background jobs, notifications
Container Runtime	Docker + Docker Compose	Local dev and staging
Orchestration	Kubernetes (prod)	Auto-scaling on cloud
CI/CD	GitHub Actions	Auto test and deploy on push
Monitoring	Prometheus + Grafana	System metrics and alerts
Error Tracking	Sentry	Frontend and backend errors
CDN	Cloudflare	Static assets, DDoS protection
Maps	Google Maps Platform	Geocoding, directions, tracking

3. Database Schema Design

All relational data uses PostgreSQL. Each service has its own schema namespace. Key tables are listed below with columns and relationships.

Table: users

id	UUID PRIMARY KEY	DEFAULT gen_random_uuid()
full_name	VARCHAR(100)	NOT NULL
phone	VARCHAR(20)	UNIQUE NOT NULL
email	VARCHAR(150)	UNIQUE
password_hash	TEXT	
role	ENUM('customer','vendor','driver','admin')	
profile_photo_url	TEXT	
is_verified	BOOLEAN	DEFAULT false
is_active	BOOLEAN	DEFAULT true
created_at	TIMESTAMP	DEFAULT NOW()
updated_at	TIMESTAMP	DEFAULT NOW()

Table: addresses

id	UUID PRIMARY KEY
user_id	UUID REFERENCES users(id)
label	VARCHAR(50) -- 'Home', 'Work', etc.
address_line	TEXT NOT NULL
city	VARCHAR(80)
latitude	DECIMAL(10,8)
longitude	DECIMAL(11,8)
is_default	BOOLEAN DEFAULT false

Table: vendors

id	UUID PRIMARY KEY
user_id	UUID REFERENCES users(id)
business_name	VARCHAR(150) NOT NULL
category	ENUM('grocery','restaurant','hotel','hall','service','beauty')
description	TEXT
logo_url	TEXT
banner_url	TEXT
phone	VARCHAR(20)
address_id	UUID REFERENCES addresses(id)
latitude	DECIMAL(10,8)
longitude	DECIMAL(11,8)

```

opening_hours JSONB -- {mon:{open:'08:00',close:'22:00'}, ...}
is_open BOOLEAN DEFAULT true
is_verified BOOLEAN DEFAULT false
avg_rating DECIMAL(3,2) DEFAULT 0
total_reviews INT DEFAULT 0
delivery_radius_km INT DEFAULT 5
min_order_amount DECIMAL(10,2)
created_at TIMESTAMP DEFAULT NOW()

```

Table: products

```

id UUID PRIMARY KEY
vendor_id UUID REFERENCES vendors(id)
name VARCHAR(200) NOT NULL
description TEXT
category VARCHAR(100)
price DECIMAL(10,2) NOT NULL
discount_price DECIMAL(10,2)
unit VARCHAR(50) -- 'kg', 'piece', 'pack'
stock_quantity INT DEFAULT 0
image_url TEXT
is_available BOOLEAN DEFAULT true
created_at TIMESTAMP DEFAULT NOW()

```

Table: orders

```

id UUID PRIMARY KEY
order_number VARCHAR(20) UNIQUE NOT NULL
customer_id UUID REFERENCES users(id)
vendor_id UUID REFERENCES vendors(id)
driver_id UUID REFERENCES users(id)
delivery_address_id UUID REFERENCES addresses(id)
status
ENUM('pending', 'confirmed', 'preparing', 'ready', 'picked_up', 'delivered', 'cancelled')
order_type ENUM('delivery', 'pickup')
subtotal DECIMAL(10,2)
delivery_fee DECIMAL(10,2)
discount_amount DECIMAL(10,2) DEFAULT 0
total_amount DECIMAL(10,2)
payment_method ENUM('card', 'wallet', 'cod')
payment_status ENUM('pending', 'paid', 'failed', 'refunded')
estimated_delivery_time INT -- minutes

```

```
special_instructions TEXT
created_at TIMESTAMP DEFAULT NOW()
```

Table: order_items

```
id UUID PRIMARY KEY
order_id UUID REFERENCES orders(id)
product_id UUID REFERENCES products(id)
quantity INT NOT NULL
unit_price DECIMAL(10,2) NOT NULL
total_price DECIMAL(10,2) NOT NULL
notes TEXT
```

Table: bookings

```
id UUID PRIMARY KEY
booking_number VARCHAR(20) UNIQUE NOT NULL
customer_id UUID REFERENCES users(id)
vendor_id UUID REFERENCES vendors(id)
booking_type ENUM('hotel', 'hall', 'beauty', 'service')
check_in_date DATE
check_out_date DATE
event_date DATE
start_time TIME
end_time TIME
guest_count INT
total_amount DECIMAL(10,2)
deposit_amount DECIMAL(10,2)
status ENUM('pending', 'confirmed', 'completed', 'cancelled')
requirements TEXT
created_at TIMESTAMP DEFAULT NOW()
```

Table: service_requests

```
id UUID PRIMARY KEY
customer_id UUID REFERENCES users(id)
assigned_provider_id UUID REFERENCES users(id)
service_type VARCHAR(100) -- 'plumber', 'electrician', 'ac_tech'
title VARCHAR(200) NOT NULL
description TEXT
address_id UUID REFERENCES addresses(id)
is_urgent BOOLEAN DEFAULT false
offered_rate DECIMAL(10,2)
```



```

status ENUM('open','assigned','in_progress','completed','cancelled')
scheduled_at TIMESTAMP
created_at TIMESTAMP DEFAULT NOW()

```

Table: drivers

```

id UUID PRIMARY KEY
user_id UUID REFERENCES users(id)
vehicle_type ENUM('bike','car','van','truck')
vehicle_number VARCHAR(20)
license_number VARCHAR(50)
is_online BOOLEAN DEFAULT false
current_latitude DECIMAL(10,8)
current_longitude DECIMAL(11,8)
last_location_update TIMESTAMP
total_deliveries INT DEFAULT 0
avg_rating DECIMAL(3,2) DEFAULT 0
earnings_today DECIMAL(10,2) DEFAULT 0

```

Table: reviews

```

id UUID PRIMARY KEY
reviewer_id UUID REFERENCES users(id)
vendor_id UUID REFERENCES vendors(id)
order_id UUID REFERENCES orders(id)
rating INT CHECK (rating BETWEEN 1 AND 5)
comment TEXT
photos JSONB -- array of image URLs
created_at TIMESTAMP DEFAULT NOW()

```

Table: payments

```

id UUID PRIMARY KEY
reference_id UUID -- order_id or booking_id
reference_type ENUM('order','booking')
user_id UUID REFERENCES users(id)
amount DECIMAL(10,2)
currency VARCHAR(10) DEFAULT 'LKR'
method ENUM('card','wallet','cod','bank_transfer')
gateway VARCHAR(50) -- 'stripe','payheredev','manual'
gateway_transaction_id TEXT
status ENUM('pending','completed','failed','refunded')
created_at TIMESTAMP DEFAULT NOW()

```

Table: notifications

id	UUID PRIMARY KEY
user_id	UUID REFERENCES users(id)
title	VARCHAR(200)
body	TEXT
type	VARCHAR(80) -- 'order_update','promo','system'
data	JSONB -- deep link params
is_read	BOOLEAN DEFAULT false
sent_at	TIMESTAMP DEFAULT NOW()

4. API Architecture & Endpoints

All APIs are RESTful, versioned at `/api/v1/`, protected via Bearer JWT. WebSocket namespace `/ws` is used for real-time events.

Base URL Structure

Production: `https://api.localconnect.app/api/v1/`

Staging: `https://staging-api.localconnect.app/api/v1/`

WebSocket: `wss://api.localconnect.app/ws`

Admin API: `https://api.localconnect.app/admin/v1/`

Core API Groups

Auth	POST <code>/auth/register</code> POST <code>/auth/login</code> POST <code>/auth/refresh</code> POST <code>/auth/logout</code> POST <code>/auth/forgot-password</code> POST <code>/auth/verify-phone</code>
Users	GET <code>/users/me</code> PATCH <code>/users/me</code> POST <code>/users/addresses</code> GET <code>/users/addresses</code> DELETE <code>/users/addresses/:id</code>
Vendors	GET <code>/vendors</code> GET <code>/vendors/:id</code> POST <code>/vendors (create)</code> PATCH <code>/vendors/:id</code> GET <code>/vendors/nearby?lat=&lng;=&radius;=</code> GET <code>/vendors/:id/products</code> PATCH <code>/vendors/:id/toggle-status</code>
Products	POST <code>/products</code> GET <code>/products/:id</code> PATCH <code>/products/:id</code> DELETE <code>/products/:id</code> PATCH <code>/products/:id/toggle-available</code> POST <code>/products/bulk-update-stock</code>
Orders	POST <code>/orders</code> GET <code>/orders/:id</code> PATCH <code>/orders/:id/status</code> GET <code>/orders/my-orders</code> GET <code>/orders/vendor/:vendorId</code> POST <code>/orders/:id/cancel</code> GET <code>/orders/:id/track</code>
Bookings	POST <code>/bookings</code> GET <code>/bookings/:id</code> PATCH <code>/bookings/:id/status</code> GET <code>/bookings/my-bookings</code> GET <code>/bookings/vendor/:vendorId</code> DELETE <code>/bookings/:id/cancel</code>
Services	POST <code>/service-requests</code> GET <code>/service-requests/nearby</code> PATCH <code>/service-requests/:id/accept</code> PATCH <code>/service-requests/:id/complete</code> GET <code>/service-requests/my-jobs</code>
Drivers	PATCH <code>/drivers/go-online</code> PATCH <code>/drivers/go-offline</code> PATCH <code>/drivers/update-location</code> GET <code>/drivers/available-jobs</code> PATCH <code>/drivers/accept-delivery/:orderId</code> GET <code>/drivers/earnings</code>
Payments	POST <code>/payments/initiate</code> POST <code>/payments/webhook</code> GET <code>/payments/:id</code> POST <code>/payments/refund</code>
Reviews	POST <code>/reviews</code> GET <code>/reviews/vendor/:vendorId</code> DELETE <code>/reviews/:id</code>
Search	GET <code>/search?q=&category;=■=&lng;=&sort;=</code> GET <code>/search/suggestions</code>
Notifications	GET <code>/notifications</code> PATCH <code>/notifications/:id/read</code> PATCH <code>/notifications/read-all</code>

WebSocket Events

Event Name	Direction	Payload Summary
order:status_update	Server → Client	orderId, newStatus, timestamp
driver:location_update	Server → Client	driverId, lat, lng, heading
order:new_order	Server → Vendor	Full order object
driver:new_job	Server → Driver	orderId, pickup, dropoff coords
service:new_request	Server → Provider	requestId, type, distance, rate
notification:new	Server → Client	title, body, type, data

5. Module 1 — Local Marketplace (Grocery & Retail)

Allows any local shop — grocery stores, supermarkets, pharmacies, bakeries, hardware shops — to list products and receive delivery orders.

Key Features

- Shop onboarding: vendor registers, adds products with photos, price, stock, unit
- Product categories: grocery, vegetables, fruits, dairy, bakery, household items, medicine
- Customer browsing: search by shop name, product name, or category; filter by distance
- Cart system: add from multiple shops, see per-shop minimum order requirement
- Order placement: choose delivery address, time slot (ASAP or scheduled), payment method
- Order management for vendor: dashboard shows pending, preparing, ready tabs
- Stock management: auto mark out-of-stock when quantity hits zero
- Bulk ordering: vendors offer discounted bulk packs for pre-orders
- Group orders: multiple customers in same area combine order to share delivery cost
- Substitution requests: vendor can suggest alternative if item is unavailable

Database Tables	vendors, products, orders, order_items, addresses
Key APIs	POST /orders, GET /vendors/nearby, GET /vendors/:id/products
App Screens	Shop listing, Shop detail, Product search, Cart, Order status, Order history

6. Module 2 — Food & Restaurant Ordering

Restaurants, cafes, cloud kitchens, and home cooks list menus and receive food delivery or dine-in pre-orders.

Key Features

- Menu builder: categories (starters, mains, desserts, drinks) with photos and prices
- Item customization: add-ons, remove ingredients, size options with price difference
- Operating hours: auto-close ordering outside restaurant hours
- Estimated prep time: vendor sets time per order, shown to customer before checkout
- Table reservation: dine-in booking with date, time, and guest count
- Scheduled ordering: order tonight for tomorrow morning breakfast delivery
- Combo deals: create meal combos at bundle price
- Live order board: kitchen display showing incoming orders in preparation queue
- Allergen tagging: mark items as contains nuts, dairy, gluten, etc.
- Popular & recommended: algorithm highlights top-ordered items

Database Tables	vendors, products, orders, order_items, bookings
Key APIs	POST /orders, GET /vendors?category=restaurant, PATCH /orders/:id/status
App Screens	Restaurant listing, Menu view, Item customizer, Cart, Tracking map, Review screen

7. Module 3 — Hotels, Rooms & Accommodation

Hotels, guesthouses, holiday villas, and individual room owners list their properties for direct booking.

Key Features

- Property listing: room types, photos, amenities, check-in rules, price per night
- Availability calendar: owner marks blocked dates; customers see real-time availability
- Instant vs request booking: owner chooses instant confirm or manual approval
- Multi-room booking: book multiple rooms for group stays
- Price rules: weekday vs weekend pricing, seasonal rates, long-stay discounts
- Photo gallery: up to 20 property and room photos with captions
- Amenities filter: WiFi, parking, AC, pool, breakfast included, etc.
- Check-in instructions: owner sends digital check-in guide after confirmation
- Deposit handling: partial deposit at booking, remainder at check-in
- Review system: post-stay reviews with star ratings per category

Database Tables	vendors, bookings, reviews, payments
Key APIs	POST /bookings, GET /vendors?category=hotel, GET /vendors/:id/availability
App Screens	Hotel search, Property detail, Room selector, Date picker, Booking confirm, My bookings

8. Module 4 — Hall & Event Venue Booking

Wedding halls, banquet rooms, function halls, outdoor grounds, and conference rooms list their spaces for event bookings.

Key Features

- Venue listing: capacity, dimensions, setup options, catering policy, parking
- Slot-based booking: morning, afternoon, full-day, multi-day blocks
- Pricing tiers: weekday vs weekend, peak season surcharge
- Package builder: venue + catering + decoration as a bundled price
- Floor plan viewer: show hall layout options (banquet, theatre, classroom)
- Deposit and cancellation policy: configurable per venue
- Contract generation: auto-generate booking agreement PDF on confirmation
- Enquiry system: customers send enquiry with requirements before booking
- Vendor add-ons: suggest linked decorators, caterers, photographers on booking
- Dashboard: calendar view of bookings, revenue chart, pending enquiries

Database Tables	vendors, bookings, payments
Key APIs	POST /bookings, GET /vendors?category=hall, PATCH /bookings/:id/status
App Screens	Hall search, Hall detail, Date/time picker, Package selector, Confirm & pay, Contract view

9. Module 5 — Event Planning & Coordination

Customers plan complete events by selecting a venue, catering, decoration, photography, entertainment, and transport — all from within the app.

Key Features

- Event wizard: step-by-step planner (date → venue → catering → decor → photo → transport)
- Vendor marketplace per category: compare caterers, decorators, photographers side by side
- Bundle booking: book all vendors under one event reference number
- Budget tracker: set event budget, see real-time spend as vendors are added
- Checklist tool: auto-generated to-do list with deadlines for event prep
- Guest list manager: add guests, track RSVPs, share event details via link
- Vendor coordination chat: group chat between customer and all event vendors
- Timeline builder: create hour-by-hour event runsheet
- Post-event invoice: consolidated bill for all vendors in one PDF
- Event templates: pre-built packages for common events (silver wedding, corporate dinner)

Database Tables	bookings, vendors, orders, payments
Key APIs	POST /bookings (multi), GET /vendors?category=event_service, GET /events/:id/summary
App Screens	Event wizard, Vendor comparison, Budget tracker, Checklist, Guest manager, Event summary

10. Module 6 — Home Services

Customers find and book verified plumbers, electricians, AC technicians, carpenters, painters, cleaners, CCTV installers, and other skilled tradespeople.

Key Features

- Service categories: plumbing, electrical, AC & appliance, carpentry, painting, cleaning, CCTV, pest control, roofing
- Urgency mode: mark request as urgent, offer higher rate for immediate response
- Location-based matching: only providers within customer's radius receive the request
- Provider profiles: photo, skills, years of experience, certifications, reviews, hourly rate
- Job request flow: customer posts job → nearby providers notified → first to accept gets job
- Price negotiation: customer sets budget, provider can counter-offer before accepting
- Arrival tracking: like driver tracking, customer sees provider's location when en-route
- Job completion: provider marks done → customer confirms → payment released
- Dispute system: customer can raise dispute with photo evidence if job unsatisfactory
- Provider earnings dashboard: daily, weekly, monthly earnings and job history

Database Tables	service_requests, users (providers), reviews, payments
Key APIs	POST /service-requests, GET /service-requests/nearby, PATCH /service-requests/:id/accept
App Screens	Service category screen, Post a request, Provider list, Provider profile, Tracking, Review

11. Module 7 — Transport & Delivery Engine

The delivery engine is the most critical component. It handles last-mile delivery for all order types and also provides ride and cargo transport.

Delivery Flow

- 1 Customer places order and selects delivery
- 2 Order Service creates order and emits order:new_order event
- 3 Delivery Service finds online drivers within radius using PostGIS geospatial query
- 4 Nearest available driver receives push notification and in-app alert
- 5 Driver accepts → status changes to 'assigned'
- 6 Driver arrives at vendor, picks up order → status 'picked_up'
- 7 Driver navigates to customer address via Google Maps API
- 8 Real-time location broadcast every 5 seconds via Socket.io to customer app
- 9 Driver marks delivered → customer confirms → payment settled → reviews unlocked
- 10 If no driver accepts within 3 min, request expands to 10km radius

Driver App Core Screens

- Go online/offline toggle with earnings summary
- Incoming job request with accept/decline (30-second timer)
- Navigation screen with embedded Google Maps
- Order detail (pickup notes, customer contact)
- Earnings history (today, week, month)
- Profile and vehicle documents

Delivery Fee Calculation

$\text{Fee} = \text{Base Fee} + (\text{Distance in km} \times \text{Per-km Rate}) + \text{Peak Hour Surcharge}$

Example: Base = LKR 50, Rate = LKR 25/km, Distance = 3km → Fee = LKR 125

Database: PostGIS for Location Queries

```
ALTER TABLE drivers ADD COLUMN location GEOGRAPHY(POINT, 4326);
CREATE INDEX idx_driver_location ON drivers USING GIST(location);
-- Find drivers within 5km:
SELECT * FROM drivers WHERE is_online=true AND ST_DWithin(location,
ST_MakePoint(lng,lat)::geography, 5000);
```

13. Three User Apps

Customer App

- Onboarding: phone number OTP login, set home address
- Home screen: category tabs (grocery, food, hotel, services), nearby shops, promotions
- Search: universal search across all vendors and products
- Cart: multi-vendor cart with per-vendor checkout or combined checkout
- Checkout: address selector, delivery time, payment method
- Live tracking: animated map showing driver with ETA
- My bookings: upcoming and past hotel/hall/service bookings
- Notifications: order updates, promotions, booking confirmations
- Profile: personal details, saved addresses, payment cards, loyalty points
- Reviews: rate vendors and drivers after service completion

Vendor / Business App

- Dashboard: today's orders, revenue, pending actions summary
- Order management: incoming orders with accept/reject, status updates
- Menu/product manager: add, edit, toggle availability, update prices
- Inventory tracker: low-stock alerts, bulk update quantities
- Booking calendar: view upcoming bookings, manage availability
- Analytics: best-selling items, peak hours, revenue trends
- Payout history: see earnings, commission deducted, transfer history
- Reviews manager: respond to customer reviews
- Profile settings: opening hours, delivery radius, minimum order
- Promotions: create discount codes or limited-time offers

Driver / Service Provider App

- Go online toggle: visible to dispatch system only when online
- Job requests: popup notification with distance, pickup, dropoff, estimated earnings
- Navigation: integrated turn-by-turn directions to pickup and dropoff
- Order details: item list, vendor contact, customer contact, special notes
- Delivery confirmation: barcode scan or OTP from customer to confirm
- Earnings tracker: today, this week, lifetime — with per-delivery breakdown
- History: all completed jobs with ratings received
- Documents: upload and manage license, insurance, vehicle docs
- Support: in-app chat with platform support team
- Ratings: see and manage customer feedback

19–21. Build Phases: Roadmap

Phase 1 — MVP Launch

Months 1–4

Goal: Get the core working: grocery ordering + food delivery + basic transport in ONE neighborhood.

Detailed Task Breakdown

- WEEK 1–2: Project setup — monorepo, Git, CI/CD pipeline, Docker dev environment
- WEEK 3–4: Auth service — phone OTP, JWT, user registration for all roles
- WEEK 5–6: Vendor onboarding — shop registration, product listing, basic dashboard
- WEEK 7–8: Customer app core — home screen, vendor listing, product browsing
- WEEK 9–10: Order service — cart, checkout, order creation, order status flow
- WEEK 11–12: Driver app — go online, job acceptance, basic navigation
- WEEK 13–14: Payment integration — cash on delivery + one card payment gateway
- WEEK 15–16: Notifications — push notifications for order events
- WEEK 16: Soft launch with 20 shops, 15 drivers in one area

Phase Deliverables

✓ Customer iOS + Android app	✓ Vendor mobile app	✓ Driver app
✓ Basic vendor web dashboard	✓ Payment processing	✓ Live order tracking

Phase 2 — Full Platform

Months 5–9

Goal: Add all remaining modules and launch across multiple vendors and categories.

Detailed Task Breakdown

- MONTH 5: Restaurant module — menu builder, prep time, kitchen display
- MONTH 5: Hotel booking module — availability calendar, booking confirmation
- MONTH 6: Hall booking module — slot booking, deposit, contract PDF
- MONTH 6: Home services module — urgent job requests, provider matching
- MONTH 7: Event planning wizard — multi-vendor event bundling
- MONTH 7: Beauty & wellness booking — time-slot appointments
- MONTH 8: Admin panel — full platform control, vendor verification, analytics
- MONTH 8: Review and rating system — post-order and post-booking
- MONTH 9: Advanced search — Elasticsearch, filters, geo-search
- MONTH 9: Promotions system — discount codes, flash sales, featured listings

Phase Deliverables

✓ All 8 modules live	✓ Admin dashboard	✓ Review system
✓ Advanced search	✓ Promo engine	✓ Full vendor dashboard web

Goal: Expand to multiple cities, add AI features, loyalty program, and enterprise tools.

Detailed Task Breakdown

- MONTH 10: Multi-city support — city selector, city-specific vendor pools
- MONTH 10: Loyalty points system — earn on every order, redeem for discounts
- MONTH 11: AI recommendations — personalized home feed based on order history
- MONTH 11: Group orders — neighbors combine orders from same shop
- MONTH 12: Scheduled deliveries — subscribe weekly for regular grocery delivery
- MONTH 12: Business analytics upgrade — vendor revenue forecasting, inventory prediction
- MONTH 13+: White-label solution — sell platform to other regions
- MONTH 13+: Enterprise accounts — corporate meal plans, bulk service contracts
- MONTH 14+: In-app advertising — vendors pay to boost listing visibility
- ONGOING: Performance optimization, load testing, security audits

Phase Deliverables

✓ Multi-city deployment	✓ AI-powered feed	✓ Loyalty program
✓ Subscription orders	✓ Enterprise accounts	✓ Platform analytics

22. Tech Stack & Infrastructure

Layer	Technology	Version/Notes
Mobile App	React Native + Expo	iOS & Android, one codebase
Web Frontend	Next.js 14 (App Router)	SSR + SSG, TypeScript
UI Library	Tailwind CSS + shadcn/ui	Component library
State Management	Zustand + React Query	Local + server state
Backend Framework	Node.js + Express.js	v20 LTS, TypeScript
ORM	Prisma	Type-safe DB queries
Primary DB	PostgreSQL 16 + PostGIS	Geospatial support
Cache / Queue	Redis 7 + BullMQ	Sessions, jobs, pub/sub
Real-time	Socket.io	Driver tracking, notifications
Search	Elasticsearch 8	Full-text + geo search
Auth	JWT + bcrypt + OTP via MSG91	Phone number login
File Storage	AWS S3 / DigitalOcean Spaces	Images, PDFs, documents
Maps	Google Maps Platform	Geocoding, Directions, Places
Payment	Stripe + PayHere (LK)	Cards + local methods
Push Notifications	Firebase Cloud Messaging	iOS, Android, Web
SMS	MSG91 / Twilio	OTP, order alerts
Email	SendGrid / AWS SES	Transactional emails
Container	Docker + Docker Compose	Dev + staging
Orchestration	Kubernetes (GKE/EKS)	Production auto-scale
CI/CD	GitHub Actions	Test + deploy on push
Monitoring	Prometheus + Grafana	Metrics + alerts
Error Tracking	Sentry	FE + BE error monitoring
CDN	Cloudflare	Static files, DDoS shield
Testing	Jest + Supertest + Cypress	Unit, API, E2E tests
API Docs	Swagger / OpenAPI 3.0	Auto-generated from code

23. Revenue Model & Business Logic

Revenue Stream	How It Works	Target Rate
Order Commission	% taken from each order total	8–15% per order
Vendor Subscription	Monthly fee for listed vendors	LKR 2,000–8,000/mo
Featured Listings	Vendors pay to appear at top of results	LKR 500–2,000/week
Delivery Fee	Per-km charge to customer on each delivery	LKR 30–150/order
Premium Customer Plan	Unlimited free delivery + priority support	LKR 1,500/month
Event Booking Fee	Flat fee on hall and hotel bookings	3–5% of booking
Service Request Cut	Platform fee on home service jobs	10% of job value
In-app Promotions	Banners and sponsored content in app	Fixed package deals

24. AI Agent Build Instructions

This section provides structured prompts and instructions for an AI coding agent (such as Claude Code, Cursor, or GitHub Copilot Workspace) to build the platform from scratch.

Project Initialization Prompt

```
You are building LocalConnect, a community super-app. Stack: Node.js + Express + TypeScript (backend), Next.js 14 + TypeScript (web), React Native + Expo (mobile). Database: PostgreSQL 16 with Prisma ORM. Cache: Redis 7. Realtime: Socket.io. Create a monorepo with this structure: /apps/web - Next.js customer web app /apps/vendor-web - Next.js vendor dashboard /apps/admin - Next.js admin panel /apps/mobile - React Native (Expo) for customer, vendor, driver /services/api - Main Express API /services/delivery - Delivery dispatch microservice /packages/db - Prisma schema + generated client /packages/types - Shared TypeScript interfaces /packages/utils - Shared utility functions Root: package.json with workspaces, turbo.json for build pipeline.
```

Database Schema Prompt

```
Using the /packages/db workspace, create the Prisma schema file with these models: User, Address, Vendor, VendorCategory (enum), Product, Order, OrderItem, OrderStatus (enum), Booking, BookingType (enum), ServiceRequest, Driver, Review, Payment, Notification, LoyaltyPoint. Enable PostGIS extension in the migration for geospatial driver queries. Add all foreign key relations, indexes on frequently queried fields (vendor_id, customer_id, status, created_at). Run: npx prisma migrate dev --name init
```

Auth Service Prompt

```
Build the authentication system in /services/api/src/modules/auth/: - Phone OTP flow: POST /auth/send-otp → send 6-digit code via MSG91 SMS API - POST /auth/verify-otp → verify code, create user if new, return JWT access token + refresh token - POST /auth/refresh → validate refresh token in Redis, return new access token - POST /auth/logout → blacklist refresh token in Redis - Middleware: authenticateToken.ts → verify JWT, attach user to req.user - Role guard middleware: requireRole('vendor' | 'driver' | 'admin') Use bcrypt for any password hashing. Store OTP codes in Redis with 5-minute TTL.
```

Order Service Prompt

```
Build the order module in /services/api/src/modules/orders/: POST /orders: 1. Validate cart items against current product availability and prices 2. Calculate subtotal, delivery fee (Base + km * rate), any applied discount 3. Create Order record with status='pending' 4. Create OrderItem records for each cart item 5. Initiate payment if method != 'cod' 6. Emit Socket.io event 'order:new_order' to vendor room 7. Trigger delivery dispatch if order confirmed PATCH /orders/:id/status: - Vendor can update: pending→confirmed, confirmed→preparing, preparing→ready - Driver can update: ready→picked_up, picked_up→delivered - Customer can: pending→cancelled (within 2 min) - Emit Socket.io event 'order:status_update' to customer room on each change
```

Delivery Dispatch Service Prompt

```
Build /services/delivery/src/dispatch.ts: When an order is ready for pickup: 1. Query PostgreSQL with PostGIS: SELECT drivers within 5km radius, is_online=true, not currently delivering 2. Sort by distance ASC 3. Send job offer to nearest driver via Socket.io event 'driver:new_job' 4. Start 30-second Redis countdown timer for driver response 5. If driver accepts: update order.driver_id, emit status update to customer 6. If timer expires: try next nearest driver (expand radius to 10km after 3 failed attempts) 7. Update driver.current_order_id in Redis for load management Driver location updates: - Driver app emits 'driver:location' every 5 seconds with lat/lng - Service stores in Redis: SET driver:{id}:location '{"lat":..,"lng":..}' EX 30 - Broadcasts to subscribed customer: 'order:{orderId}:driver_location'
```

React Native App Structure Prompt

```
In /apps/mobile, set up Expo Router for navigation: app/(auth)/login.tsx - Phone input + OTP verification app/(customer)/index.tsx - Home feed with categories app/(customer)/search.tsx - Universal search app/(customer)/cart.tsx - Cart management app/(customer)/checkout.tsx - Address, time, payment selection app/(customer)/track/[orderId].tsx - Live map tracking screen app/(customer)/bookings.tsx - My bookings list app/(vendor)/orders.tsx - Incoming orders app/(vendor)/menu.tsx - Product management app/(driver)/jobs.tsx - Available + active jobs app/(driver)/navigate/[orderId].tsx - Turn-by-turn navigation Use Zustand for auth state and cart state. Use React Query for all API calls with automatic retry and cache. Use react-native-maps for the tracking screen with animated driver marker. Use Expo Notifications for push notification handling.
```

Security Checklist Prompt

- All endpoints behind authenticateToken middleware except /auth/* routes
- Input validation using Zod on every request body and query param
- Rate limiting: 100 req/15min per IP on auth routes, 500 req/15min on API routes (Redis-backed)
- SQL injection prevention: use Prisma parameterized queries only, never raw string interpolation
- File upload validation: check MIME type + file size limit (5MB) before uploading to S3
- CORS: whitelist only known frontend domains
- Helmet.js: set security headers on all responses
- Sensitive env vars: store in .env, never commit — use GitHub Secrets in CI/CD
- Payment webhooks: verify Stripe/PayHere signature on every webhook request
- Admin routes: additional requireRole('admin') guard + IP whitelist option

25. Testing, QA & Launch Checklist

Backend

- All API endpoints return correct HTTP status codes
- JWT expiry and refresh flow tested
- Order state machine tested for all valid and invalid transitions
- Payment webhook tested with Stripe test events
- Delivery dispatch tested with mock driver locations
- Rate limiting verified on auth and API routes
- Database indexes verified with EXPLAIN ANALYZE on slow queries
- All sensitive fields (password, card numbers) never returned in API responses

Mobile App

- OTP login works on both iOS and Android
- Push notifications received when app is in background
- Live tracking map updates smoothly every 5 seconds
- Cart persists across app restarts (AsyncStorage)
- Payment card input meets PCI DSS requirements (use Stripe SDK, never store raw)
- Camera permission for photo upload handled gracefully
- Offline state shows appropriate message
- Deep links from notifications navigate to correct screen

Vendor Dashboard

- Incoming order notification received within 3 seconds
- Order accept/reject works correctly
- Menu toggle (available / unavailable) reflects immediately in customer app
- Analytics charts load correctly with real data
- Payout history shows correct amounts after commission deduction

Performance

- API response time under 200ms for 95% of requests under load
- WebSocket supports 1,000 concurrent connections

- Elasticsearch search returns results under 100ms
- Mobile app initial load under 3 seconds on 4G
- Images served via CDN, compressed to WebP format

Pre-Launch

- App submitted to App Store and Google Play — approval received
- Domain, SSL certificate, and Cloudflare configured
- Production database backed up and restore tested
- Monitoring alerts configured for API downtime and error spikes
- On-call plan defined for launch week
- 20+ vendors onboarded and verified
- 15+ drivers registered, trained, and online
- Payment gateway live keys activated and tested with real transaction
- Support contact (WhatsApp/email) listed in app

LocalConnect — Full Implementation Master Plan v1.0 Ready to build. Start with Phase 1 MVP. Ship to production in 4 months.